

EXAMINATION ASSIGNMENT

Multi-Database Entity Resolution & Unified Data Repository

Technical Development Evaluation

1. Assignment Objective

Build a complete web-based data integration and entity-resolution platform that can ingest multiple large databases, understand their structures, clean and normalize their fields, identify matching records across sources, and build a unified master repository.

The central challenge is not simply importing databases. The system must understand that the same real-world entity may appear differently across multiple datasets and progressively connect available information.

2. Core Winning Feature: Progressive Entity Enrichment

This is the primary concept of the assignment.

When the user searches for an identifier such as an email address, the system should search across all available datasets. If the matching record contains additional identifiers such as a mobile number, username, member ID, company name or address, the system should use those newly discovered identifiers to search the remaining datasets.

Example flow:

1. User searches: john@example.com
2. Database A finds John Doe and mobile number 9876543210.
3. The system uses 9876543210 as another matching identifier.
4. Database B contains that mobile number and provides an address.
5. Database C contains the same person under a different field name and provides company information.
6. The system combines the discovered information into a unified entity.
7. The system retains the source of every discovered record.

The system should continue this enrichment process until no meaningful new matching information can be discovered.

3. Data Sources

Students may use publicly available datasets for the examination. Suitable examples include government datasets, Startup India data, HR-style datasets, architecture/business datasets, open-data repositories, or other legally downloadable datasets.

The exact subject matter is not important. The important requirement is to demonstrate multiple datasets with different structures and overlapping entities.

Students should create or obtain enough data to demonstrate meaningful matching. Synthetic/static data may also be generated where necessary to guarantee known matches.

4. Required Database Formats

The primary database input should support:

- ⌚ SQL databases / SQL dumps
- ⌚ CSV files

Students should design the import process so that a large database can also be uploaded or processed in multiple parts.

Example:

- ⌚ Database 1 - Part 1
- ⌚ Database 1 - Part 2
- ⌚ Database 1 - Part 3

The system should understand that these are parts of a dataset where the user identifies them as such, and should process them without unnecessarily creating duplicate entities.

5. Large Dataset Requirement

The architecture must be designed for large datasets. The expected working scale is more than 100,000 records, with datasets potentially reaching 500,000, 1,000,000, 2,000,000 or more records.

Students should not assume that the entire dataset can be loaded into browser memory or processed in one request.

- ⌚ Use batch processing.
- ⌚ Use database indexes appropriately.
- ⌚ Process large imports asynchronously or in manageable jobs.
- ⌚ Avoid loading millions of rows into application memory at once.
- ⌚ Show processing progress.
- ⌚ Allow processing to continue after a page refresh.

6. Database Import Workflow

8. Create a database/source entry.
9. Upload or connect the SQL/CSV source.
10. Read the available tables and columns.
11. Analyze the structure.
12. Suggest field mappings.
13. Allow the user to review or modify mappings.
14. Normalize the data.
15. Create searchable indexes.
16. Process matching against existing repositories.
17. Create or enrich master entities.
18. Record source and processing information.

7. Automatic Field Mapping

Different databases may use different names for the same concept.

Source Column	Possible Meaning	Canonical Field
email	Email address	email

email_id	Email address	email
email_address	Email address	email
mobile_number	Phone/mobile	phone
contact_no	Phone/mobile	phone
full_name	Person name	name
member_id	Entity/member identifier	member_id

The system should use AI or another intelligent mapping mechanism to suggest canonical field mappings. A small/fast AI API may be used for this purpose.

The user must be able to review and correct the suggested mapping before or during processing.

8. Data Cleaning and Normalization

Cleaning is a core requirement. Raw values must be normalized before matching.

- 🕒 Convert email addresses to a consistent case.
- 🕒 Trim leading/trailing whitespace.
- 🕒 Normalize repeated spaces.
- 🕒 Normalize phone numbers by removing irrelevant spaces and formatting characters where appropriate.
- 🕒 Normalize country/phone prefixes where the available data allows it.
- 🕒 Normalize common textual variations.
- 🕒 Handle blank/null values consistently.
- 🕒 Identify malformed values.
- 🕒 Preserve the original raw value for traceability.

The system should retain both the original value and the normalized value where practical.

9. Matching Engine

The system must match records across databases using a configurable hierarchy of identifiers.

A possible hierarchy is:

19. Exact email match
20. Exact normalized phone match
21. Username match
22. Member/customer/user identifier match
23. Other configured unique identifiers
24. Additional matching rules where confidence is sufficiently high

The matching logic should be designed so that the hierarchy can be improved or extended rather than hardcoded into a single query.

10. Master Entity Repository

When records are determined to represent the same entity, the system should create or update a master entity.

Example:

```
Master Entity #1042
Name: John Doe
Email: john@example.com
Phone: 9876543210
Username: johndoe
Address: Mumbai
```

Company: ABC Pvt Ltd

Sources:

- HR Database
- Startup Database
- Business Directory

The master repository should preserve the relationship between the unified entity and its original source records.

11. Source-Level Traceability

Every imported record must retain information about where it came from.

- 🕒 Source database
- 🕒 Source table
- 🕒 Source record/row identifier where available
- 🕒 Import batch
- 🕒 Original field name
- 🕒 Original value
- 🕒 Normalized value
- 🕒 Processing timestamp

Where possible, individual master fields should also retain source information so the user can understand why a value exists.

12. Progressive Search and Enrichment

Searching must not stop after the first match.

Example:

25. Search email.
26. Find matching records.
27. Extract phone, username, member ID and other useful identifiers.
28. Search using those identifiers.
29. Find additional records.
30. Extract newly discovered identifiers.
31. Search again.
32. Merge newly discovered information into the master entity.
33. Stop when no meaningful new information is discovered.

This progressive enrichment process is the main feature of the assignment.

13. New Database Processing

The platform must continue to improve as new datasets are added.

- 🕒 A new database should appear in the source/data management area.
- 🕒 Its tables and columns should be detected.
- 🕒 Field mapping should be suggested.
- 🕒 The database should be cleaned and indexed.
- 🕒 Existing master entities should be checked against the new source.
- 🕒 New matching information should enrich existing entities.
- 🕒 Unmatched records should be added as new entities or retained as unresolved records for future matching.

- ⌚ Previously processed databases should not unnecessarily be reprocessed from scratch.

14. No-Match Handling

If a record does not match any existing entity, the system should not discard it.

- ⌚ Create a new master entity where sufficient information exists.
- ⌚ Store the original source information.
- ⌚ Mark the entity as newly created.
- ⌚ Allow future datasets to match against it.

A previously unmatched record may become matchable after a new database is imported.

15. Multi-Table Relationship Understanding

Some SQL databases may contain many related tables. For example:

```
members.member_id
      ↓
member_details.member_id
      ↓
member_contacts.member_id
```

Students should consider how the system can detect and use relationships between tables, foreign keys, IDs and other linking fields.

16. Bonus: Automatic Relationship Discovery

Automatically discovering relationships between multiple tables is a bonus feature.

- ⌚ Detect foreign-key relationships.
- ⌚ Infer relationships where explicit foreign keys are missing.
- ⌚ Identify shared identifiers across tables.
- ⌚ Build a relationship map.
- ⌚ Use those relationships to enrich master entities.

17. Data Processing Status

The system should make processing status visible.

Example:

- ⌚ Uploaded
- ⌚ Inspecting structure
- ⌚ Field mapping
- ⌚ Cleaning
- ⌚ Indexing
- ⌚ Matching
- ⌚ Enriching entities
- ⌚ Completed
- ⌚ Completed with errors

The user should be able to see what is happening instead of waiting on an unexplained loading screen.

18. Repository and Search Interface

The web interface should allow a non-technical user to:

- ⌚ Add a database/source
- ⌚ View available databases
- ⌚ View tables and columns
- ⌚ Review field mappings
- ⌚ Start or resume processing
- ⌚ Search the unified repository
- ⌚ Search by email
- ⌚ Search by phone
- ⌚ Search by username
- ⌚ View all discovered information for an entity
- ⌚ View source databases
- ⌚ View matching records
- ⌚ View processing status

19. Search Result Example

A search for an email should produce an entity-oriented result rather than simply returning one database row.

Search: john@example.com

MASTER ENTITY

Name: John Doe
Email: john@example.com
Phone: 9876543210
Username: johndoe
Address: Mumbai

Matched Sources: 4

Source A: HR Database
Source B: Business Directory
Source C: Startup Database
Source D: Contact Database

20. Processing Statistics

Maintain persistent statistics for imports and matching.

- ⌚ Total databases
- ⌚ Total tables
- ⌚ Total records
- ⌚ Records processed
- ⌚ Records matched
- ⌚ New entities created
- ⌚ Entities enriched
- ⌚ Duplicate records detected
- ⌚ Unresolved records
- ⌚ Processing failures
- ⌚ Processing time
- ⌚ Last processed time

21. Performance and Caching

The system must be designed for large-scale searching and repeated matching. Students should identify performance bottlenecks and propose appropriate solutions.

Redis or another caching mechanism may be used to improve repeated lookups and job processing.

The system should use indexes and efficient queries rather than repeatedly scanning entire multi-million-record datasets.

22. Architecture Requirement

Students must provide a brief architecture document explaining how their system handles:

- 🕒 Database ingestion
- 🕒 Large-file processing
- 🕒 Field mapping
- 🕒 Data normalization
- 🕒 Matching
- 🕒 Entity enrichment
- 🕒 Master repository
- 🕒 Source traceability
- 🕒 Background processing
- 🕒 Caching
- 🕒 Error handling
- 🕒 Future database additions

The architecture should explain how the system can continue to scale as more databases are added.

23. Technology

Students may use a suitable technology stack. PHP is acceptable for the web layer, with Python or another suitable language for data processing if required.

Students may use AI tools, frameworks, libraries and documentation during development. The final system must demonstrate their own understanding of the architecture and implementation.

24. Mandatory Dataset Demonstration

The demonstration should contain multiple datasets with intentionally different schemas.

For example:

Database	Column Example	Overlap
Database A	email, full_name, mobile_number	Email + phone
Database B	email_id, name, contact_no, address	Email
Database C	username, phone, company	Phone
Database D	member_id, email, username	Email + username

The demonstration data should make it possible to show progressive enrichment across more than one source.

25. Mandatory Demonstration Flow

34. Import multiple databases.

35. Show different column names representing similar fields.
36. Run AI-assisted field mapping.
37. Show the mapping before processing.
38. Run data cleaning/normalization.
39. Process the datasets.
40. Search for an identifier such as an email.
41. Show the initial match.
42. Show newly discovered identifiers.
43. Show the system using those identifiers to discover additional records.
44. Show the final unified master entity.
45. Show the source of the discovered information.
46. Add another database.
47. Process the new database.
48. Show existing entities being enriched or new entities being created.
49. Show that duplicate records are not unnecessarily duplicated.

26. Bonus Features

The following features can demonstrate additional depth:

- 🕒 Automatic discovery of table relationships.
- 🕒 Automatic foreign-key/relationship inference.
- 🕒 Graph-based entity relationship visualization.
- 🕒 Advanced fuzzy matching.
- 🕒 Confidence scoring for matches.
- 🕒 Automatic conflict resolution between different source values.
- 🕒 Redis caching.
- 🕒 Distributed/background workers.
- 🕒 Automatic database schema documentation.
- 🕒 Automatic database profiling and quality reports.
- 🕒 Advanced search and filtering.
- 🕒 Entity history/timeline.
- 🕒 Automatic reprocessing when a new database is added.
- 🕒 Incremental processing of very large database parts.
- 🕒 Automatic detection of duplicate datasets.

27. Reliability Requirements

- 🕒 The system should not load very large datasets into browser memory.
- 🕒 Large imports should be processed in batches.
- 🕒 Long-running jobs should not depend on an open browser tab.
- 🕒 Failures should be logged and visible.
- 🕒 Processing should be resumable where practical.
- 🕒 The system should preserve raw source data or enough source information for traceability.
- 🕒 The system should not silently discard unmatched records.
- 🕒 The system should remain usable while large background jobs are processing.

28. Live Hosting

The completed system must be hosted online and available for evaluation.

- 🕒 The evaluator should be able to access the web interface without creating an account.

- ⌚ Do not password-protect the evaluation interface.
- ⌚ The live environment should contain the demonstration datasets.
- ⌚ The system should remain available during evaluation.
- ⌚ The evaluator must be able to perform searches and inspect the resulting unified entities.

29. Final Submission

- ⌚ Live hosted application URL
- ⌚ Demo login-free environment
- ⌚ Multiple sample databases
- ⌚ Sample SQL/CSV datasets used for demonstration
- ⌚ Architecture explanation
- ⌚ Field mapping demonstration
- ⌚ Matching/enrichment demonstration
- ⌚ Sample master entities
- ⌚ Processing statistics
- ⌚ Video demonstrating the complete workflow
- ⌚ List of technologies and libraries used
- ⌚ Bonus features implemented, if any

30. Key Evaluation Principle

The objective is to demonstrate a system that can turn disconnected datasets into a connected, searchable and continuously improving repository.

The most important capability is progressive entity enrichment: starting with one identifier, discovering additional identifiers, using those identifiers to search other sources, and continuously building a richer master entity while preserving source-level traceability.

A simple database importer is not sufficient. The system should demonstrate ingestion, mapping, cleaning, matching, enrichment, repository management, large-data processing and intelligent search.